

Concurrency Models

Liting Zhang

University of West Florida

Overview

- Why concurrency: performance vs responsiveness
- Shared-memory threads: races and coordination
- Synchronization: locks, semaphores, monitors, condition variables
- Deadlocks: how they happen, how to avoid them
- Message passing and actors; Go goroutines and channels
- Async/await and event loops (Node.js, TS/JS, Python async)
- Industry angle: web servers, microservices, Node back-ends, Go services
- Rust cameo: safety story and tradeoffs

Learning goals

After this lecture, you should be able to

- Explain why systems use concurrency for throughput and for latency
- Identify shared-state hazards: race conditions and lost updates
- Explain deadlock conditions and recognize common deadlock patterns
- Choose between mutexes, semaphores, and monitors for coordination
- Describe message passing and actor-style concurrency at a high level
- Explain event loops and `async/await` scheduling
- Compare thread-based concurrency and `async` I/O concurrency
- Read small examples in Java/C#, Go, TypeScript, Python `async`, and Rust

Motivation and hazards

- 1 Motivation and hazards
- 2 Threads and synchronization
- 3 Deadlocks and pitfalls
- 4 Message passing and Go
- 5 Async/await and event loops
- 6 Industry and language tradeoffs
- 7 Review

Why concurrency exists

Two common motivations:

- Performance: increase throughput by using multiple cores
- Responsiveness: keep a system interactive while work happens

Examples:

- web server handling many requests at once
- UI app keeping the interface responsive while fetching data
- pipeline processing logs while continuing to accept new events

Throughput vs latency

Throughput focus:

- goal: maximize completed work per unit time
- common in batch jobs, data processing, server farms

Latency focus:

- goal: reduce time to first response for each task
- common in interactive apps and user-facing services

Many systems want both, and must trade off carefully.

Concurrency vs parallelism

Concurrency:

- structuring the program to handle multiple tasks in progress
- tasks may interleave, even on a single core

Parallelism:

- actually running tasks at the same time on multiple cores

A program can be concurrent without being parallel.

Race condition

Race condition:

- program behavior depends on timing or interleaving of tasks
- two correct-looking pieces of code can break when combined

Common symptom:

- lost updates: two tasks overwrite each other

Concurrency bugs are often:

- rare
- timing dependent
- hard to reproduce and debug

Practice 1: classify the motivation

For each scenario, decide whether the main motivation is throughput, latency, or both.

- A web server handling 10,000 concurrent connections
- A photo editor exporting a large image while UI stays responsive
- A nightly ETL job processing 2 TB of logs
- A multiplayer game server handling physics and networking

Practice 1 Answer: classify the motivation

One reasonable classification:

- web server: both
- photo editor: latency/responsiveness
- ETL job: throughput
- game server: both, with strong latency requirements

Threads and synchronization

- 1 Motivation and hazards
- 2 Threads and synchronization
- 3 Deadlocks and pitfalls
- 4 Message passing and Go
- 5 Async/await and event loops
- 6 Industry and language tradeoffs
- 7 Review

Threads and shared memory model

Threads share memory.

- multiple threads access the same variables and objects
- coordination is required to preserve invariants

Benefits:

- direct sharing can be fast
- fits well with in-memory data structures

Costs:

- correctness depends on interleavings
- shared mutable state is the main source of bugs

Java: lost update example

Two threads increment a shared counter.

```
1 class Counter {
2     int value = 0;
3     void inc() { value = value + 1; }
4 }
5 public class Demo {
6     static Counter c = new Counter();
7     static Runnable work = () -> {
8         for (int i = 0; i < 1_000_000; i++) c.inc();
9     };
10
11     public static void main(String[] args) throws Exception {
12         Thread t1 = new Thread(work);
13         Thread t2 = new Thread(work);
14         t1.start(); t2.start();
15         t1.join(); t2.join();
16         System.out.println(c.value);
17     }
18 }
```

Expected: 2,000,000. Actual: often less.

Why increment is not atomic

The increment expands into steps:

- load value
- add 1
- store result

Two threads can interleave:

- both load the same old value
- both store $\text{old}+1$
- one increment is lost

Java: fix with synchronized

synchronized makes the critical section mutually exclusive.

```
1 | class Counter {  
2 |     int value = 0;  
3 |     synchronized void inc() { value = value + 1; }  
4 | }
```

Correctness improves, but contention can hurt performance.

Synchronization toolbox

Common primitives:

- mutex / lock: mutual exclusion for critical sections
- semaphore: limit concurrency or represent resource counts
- monitor: lock plus condition waiting
- condition variable: block until a predicate becomes true

Many bugs come from:

- missing locks
- wrong lock scope
- inconsistent lock ordering
- waiting without a loop

Mutex / lock

Mutex:

- allows one thread at a time into a critical section
- protects invariants on shared mutable data

Practical guidance:

- keep critical sections small
- lock around an invariant, not around a single line
- document what the lock protects

Semaphore

Semaphore:

- a counter controlling how many threads may proceed
- `acquire()` decrements, `release()` increments

Common uses:

- limit concurrency (max N in-flight tasks)
- resource pools (N connections, N workers)

Monitor and condition variable

Monitor:

- a lock plus a way to wait for conditions safely
- wait releases the lock, then blocks, then reacquires before returning

Condition variable:

- used with a predicate and a loop
- avoids busy waiting

Java: bounded buffer sketch with wait/notify

Producer/consumer using a monitor.

```
1  import java.util.*;
2
3  class BoundedQueue<T> {
4      private final int cap;
5      private final ArrayDeque<T> q = new ArrayDeque<>();
6
7      BoundedQueue(int cap) { this.cap = cap; }
8
9      public synchronized void put(T x) throws InterruptedException {
10         while (q.size() == cap) wait();
11         q.addLast(x);
12         notifyAll();
13     }
14
15     public synchronized T take() throws InterruptedException {
16         while (q.isEmpty()) wait();
17         T x = q.removeFirst();
18         notifyAll();
19         return x;
20     }
21 }
```

while is required because conditions may change before the lock is reacquired.

Deadlocks and pitfalls

- 1 Motivation and hazards
- 2 Threads and synchronization
- 3 Deadlocks and pitfalls**
- 4 Message passing and Go
- 5 Async/await and event loops
- 6 Industry and language tradeoffs
- 7 Review

Deadlock pattern

Deadlock:

- threads wait forever because each is waiting for another to release something

Simple pattern:

- Thread A holds lock L1, wants L2
- Thread B holds lock L2, wants L1

The program may appear correct until it freezes under a rare interleaving.

Deadlock conditions (Coffman)

Deadlock becomes possible when all are present:

- mutual exclusion
- hold and wait
- no preemption
- circular wait

Avoidance strategies remove at least one condition.

Java: deadlock example

Two locks, opposite acquisition order.

```
1 final Object a = new Object();
2 final Object b = new Object();
3
4 Thread t1 = new Thread(() -> {
5     synchronized (a) {
6         try { Thread.sleep(10); } catch (Exception e) {}
7         synchronized (b) { System.out.println("t1"); }
8     }
9 });
10
11 Thread t2 = new Thread(() -> {
12     synchronized (b) {
13         try { Thread.sleep(10); } catch (Exception e) {}
14         synchronized (a) { System.out.println("t2"); }
15     }
16 });
17 t1.start(); t2.start();
```

A common fix is consistent lock ordering.

Avoiding deadlocks

Practical techniques:

- lock ordering: always acquire locks in a consistent global order
- timeouts: abandon and retry rather than wait forever
- reduce lock scope and avoid nested locks
- prefer higher-level designs: queues, actors, channels

Other common pitfalls

Issues that look like deadlocks or cause similar outages:

- blocking I/O while holding a lock
- waiting without a loop on a condition variable
- unbounded queues causing memory blowups
- priority inversion: low priority thread holds a lock needed by high priority work

In production, symptoms often appear as latency spikes and request timeouts.

Practice 3: deadlock diagnosis

In the deadlock example:

- Which Coffman conditions are present
- Give one minimal code/design change to reduce risk

Practice 3 Answer: deadlock diagnosis

Conditions present:

- mutual exclusion: locks are exclusive
- hold and wait: each thread holds one lock while waiting for the other
- no preemption: locks are released only voluntarily
- circular wait: A waits for B and B waits for A

Minimal change:

- consistent lock ordering (both threads acquire a then b)

Message passing and Go

- 1 Motivation and hazards
- 2 Threads and synchronization
- 3 Deadlocks and pitfalls
- 4 Message passing and Go**
- 5 Async/await and event loops
- 6 Industry and language tradeoffs
- 7 Review

Message passing model

Message passing:

- components communicate by sending messages
- shared memory is minimized or avoided
- state is owned by one component and updated sequentially

Benefits:

- fewer shared-memory races
- clearer ownership boundaries

Costs:

- messaging overhead and protocol design
- must handle ordering, retries, and failures in distributed cases

Actor model intuition

Actor:

- owns its state
- processes one message at a time
- can send messages to other actors

Typical result:

- fewer data races in local actor state
- concurrency scales by adding more actors

Similar ideas appear in event-driven microservices and queue-based systems.

Channels as coordination

Channel:

- a queue connecting producers and consumers
- can be buffered or unbuffered
- can provide backpressure by blocking senders when full

Channels often simplify reasoning:

- send/receive is explicit coordination
- avoid many shared-memory locks

Go: goroutines and channels

Goroutines are lightweight concurrent functions.

```
1 package main
2 import "fmt"
3 func worker(in <-chan int, out chan<- int) {
4     for x := range in {
5         out <- x * x
6     }
7     close(out)
8 }
9 func main() {
10     in := make(chan int)
11     out := make(chan int)
12     go worker(in, out)
13     go func() {
14         for _, v := range []int{1, 2, 3, 4} {
15             in <- v
16         }
17         close(in)
18     }()
19     for y := range out {
20         fmt.Println(y)
21     }
22 }
```

The worker reads until in is closed, then closes out.

Go: worker pool pattern

A common production pattern:

- a bounded number of worker goroutines
- a jobs channel
- a results channel

Why it matters:

- bounds resource usage
- provides backpressure under overload
- avoids spawning unbounded goroutines

Practice 4: detect a channel deadlock

What happens here?

```
1 package main
2
3 func main() {
4     ch := make(chan int)
5     ch <- 1
6 }
```

- Does it finish, block, or crash
- Why

Practice 4 Answer: detect a channel deadlock

It blocks forever.

- ch is unbuffered
- send blocks until a receiver is ready
- there is no receiver

Async/await and event loops

- 1 Motivation and hazards
- 2 Threads and synchronization
- 3 Deadlocks and pitfalls
- 4 Message passing and Go
- 5 Async/await and event loops**
- 6 Industry and language tradeoffs
- 7 Review

Event loop model

Event loop:

- one main thread processes queued tasks/callbacks
- I/O registers completion events, then returns quickly
- when I/O completes, a callback/task is queued

Benefits:

- handles many concurrent I/O tasks efficiently
- reduces shared-memory races when most code runs on one thread

Cost:

- blocking the event loop breaks responsiveness for everything

Async/await as structured callbacks

async/await:

- syntax that makes promise/callback style code readable
- await yields control back to the scheduler/event loop

Operational picture:

- run until first await
- pause the function
- resume later when the awaited operation completes

TypeScript: basic async/await

```
1 function delay(ms: number): Promise<void> {  
2     return new Promise(resolve => setTimeout(resolve, ms));  
3 }  
4  
5 async function f(): Promise<void> {  
6     console.log("A");  
7     await delay(10);  
8     console.log("B");  
9 }  
10  
11 console.log("start");  
12 f();  
13 console.log("end");
```

Typical output: start, A, end, then later B.

TypeScript: sequential vs parallel awaits

```
1 function work(x: number): Promise<number> {  
2     return new Promise(resolve => setTimeout(() => resolve(x *  
   ↪ x), 10));  
3 }  
4  
5 async function sequential(xs: number[]): Promise<number[]> {  
6     const out: number[] = [];  
7     for (const x of xs) out.push(await work(x));  
8     return out;  
9 }  
10  
11 async function parallel(xs: number[]): Promise<number[]> {  
12     return Promise.all(xs.map(work));  
13 }
```

parallel starts all operations first, then awaits completion.

Python: async/await basics

```
1 import asyncio
2
3 async def work(x: int) -> int:
4     await asyncio.sleep(0.01)
5     return x * x
6
7 async def main() -> None:
8     xs = [1, 2, 3, 4]
9     ys = await asyncio.gather(*(work(x) for x in xs))
10    print(ys)
11
12 asyncio.run(main())
```

gather is similar to Promise.all.

Async does not imply CPU parallelism

Many async runtimes:

- overlap I/O waits
- run callbacks on one main thread

Consequence:

- CPU-heavy code blocks the event loop
- for CPU-heavy work, use worker pools, threads, or processes

This is why production Node services often offload CPU-heavy tasks.

Practice 5: predict output ordering (TypeScript)

Predict the order.

```
1 function delay(ms: number): Promise<void> {  
2     return new Promise(r => setTimeout(r, ms));  
3 }  
4  
5 async function g(): Promise<void> {  
6     console.log("g1");  
7     await delay(0);  
8     console.log("g2");  
9 }  
10  
11 console.log("A");  
12 g();  
13 console.log("B");
```

Practice 5 Answer: predict output ordering (TypeScript)

A typical order:

- A
- g1
- B
- g2

Reason:

- g runs until await, then yields
- g2 runs later when the promise resolves

Industry and language tradeoffs

- 1 Motivation and hazards
- 2 Threads and synchronization
- 3 Deadlocks and pitfalls
- 4 Message passing and Go
- 5 Async/await and event loops
- 6 Industry and language tradeoffs**
- 7 Review

Choosing a concurrency model in practice

Questions engineers ask:

- Is the workload I/O-heavy or CPU-heavy
- How many concurrent tasks are typical at peak
- What are latency requirements and tail latency constraints
- What failure modes matter: overload, backpressure, partial outages

In practice, systems often mix models:

- async I/O for network calls
- worker pools for CPU-bound transformations
- message queues for cross-service decoupling

Web servers: typical concurrency choices

Common architectures:

- thread-per-request: simple mental model, heavy at high scale
- thread pool: bound thread count, queue work
- event loop: many I/O connections, fewer threads
- hybrid: event loop plus worker pool

The best choice depends on workload shape and performance goals.

Backpressure and overload

Under load:

- unbounded queues increase memory usage and latency
- downstream services become bottlenecks

Practical mechanisms:

- bounded queues and worker pools
- semaphores limiting in-flight work
- timeouts, retries with jitter, circuit breakers

Rust cameo: safety angle

Rust emphasizes safety against data races:

- ownership and borrowing prevent many unsafe shared mutations
- safe shared mutation typically uses synchronization wrappers

Takeaway:

- language design can move concurrency errors from runtime bugs to compile-time constraints

Rust: shared counter sketch

Arc + Mutex is a common safe pattern.

```
1 use std::sync::{Arc, Mutex};
2 use std::thread;
3
4 fn main() {
5     let c = Arc::new(Mutex::new(0));
6     let mut handles = vec![];
7
8     for _ in 0..2 {
9         let c2 = Arc::clone(&c);
10        handles.push(thread::spawn(move || {
11            for _ in 0..1000 {
12                let mut v = c2.lock().unwrap();
13                *v += 1;
14            }
15        }));
16    }
17
18    for h in handles { h.join().unwrap(); }
19    println!("{}", *c.lock().unwrap());
```

Practice 6: choose a model

For each service, choose a reasonable default concurrency model and explain why:

- chat server that mostly does network I/O
- image resizing service that is CPU-heavy
- API gateway that calls many downstream services
- log processor that does batch transformations nightly

Practice 6 Answer: choose a model

One reasonable set of choices:

- chat server: event loop or async I/O, worker pool for CPU tasks if needed
- image resizing: threads/processes for CPU parallelism
- API gateway: async I/O, bounded concurrency, timeouts
- batch log processor: parallel map/reduce style, worker pools, distributed if needed

Review

- 1 Motivation and hazards
- 2 Threads and synchronization
- 3 Deadlocks and pitfalls
- 4 Message passing and Go
- 5 Async/await and event loops
- 6 Industry and language tradeoffs
- 7 Review

Summary: what to remember

- Concurrency targets throughput and responsiveness
- Shared-memory threads need synchronization to prevent races
- Locks, semaphores, and monitors solve different coordination problems
- Deadlocks come from circular waits and inconsistent acquisition patterns
- Message passing and channels isolate state and reduce shared-memory bugs
- Event loops and `async/await` excel for I/O-heavy concurrency
- Real systems mix models and need backpressure under overload
- Languages influence which concurrency patterns are idiomatic

Review questions

- Give one throughput example and one latency example for concurrency
- Why is $\text{value} = \text{value} + 1$ unsafe across threads
- What is a simple deadlock avoidance strategy
- When is a semaphore a better fit than a mutex
- Why can async code still be single-threaded
- What problem do channels solve compared to shared state